IRSTI 50.41.25
UDC 004.42:004.65

PERFORMANCE STUDY OF A SPRING BOOT REST API AND EVALUATION OF REDIS CACHING UNDER HIGH LOAD

Serik Zholdygaliev*, Ravil Yermukhan, Kulush Askhat
International Information Technology University, Almaty, Kazakhstan
E-mail: sikozholdigaliev@gmail.com, r.yermukhan@gmail.com, finf0094@gmail.com
**Corresponding author*

Abstract. Redis caching is often recommended for read-heavy REST APIs, but its benefit is rarely measured. This article evaluates Redis cache-aside in a Spring Boot 3.5 API backed by PostgreSQL 16. Seventy-five randomised paired runs compared no cache with Redis using 100 ms and 1 s client timeouts at 100, 500 and 1000 virtual users on hot keys, and with the 1 s variant on uniform keys; the service had one vCPU and a JIT-aware warm-up. On hot keys Redis removed 91 % of SQL calls and (1 s timeout) cut 99th-percentile latency by 32–61 %, but mean throughput did not change reliably (ratio 0.99 at 1000 users); on uniform keys throughput fell by 22–34 %. CPU accounting explains this: the database used only 32 % of request CPU time, while the cache path added JVM work. The hypothesised 60 % latency reduction and threefold throughput gain were not supported.

Keywords: Spring Boot, Redis, cache-aside, REST API, load testing, tail latency, JIT warm-up.

SPRING BOOT REST API ӨНІМДІЛІГІН ЗЕРТТЕУ ЖӘНЕ ЖОҒАРЫ ЖҮКТЕМЕ КЕЗІНДЕ REDIS КЭШТЕУІН БАҒАЛАУ

Serik Zholdygaliev*, Ravil Yermukhan, Kulush Askhat
Халықаралық ақпараттық технологиялар университеті, Алматы, Қазақстан
E-mail: sikozholdigaliev@gmail.com, r.yermukhan@gmail.com, finf0094@gmail.com
**Хат-хабар жүргізуге жауапты автор*

Аңдатпа. Redis кэштеуі деректерді көп оқитын REST API үшін жиі ұсынылады, бірақ оның нақты конфигурациядағы пайдасы сирек өлшенеді. Мақалада PostgreSQL 16 дерекқорымен жұмыс істейтін Spring Boot 3.5 API-індегі Redis cache-aside тәсілі бағаланады. 75 рандомизацияланған жұптық өлшеуден тұратын бақыланатын экспериментте кэшсіз нұсқа клиенттік күту уақыты 100 мс және 1 с болатын Redis нұсқаларымен 100, 500 және 1000 параллель виртуалды пайдаланушы кезінде «танымал кілттер» жүктемесінде, ал біркелкі жүктемеде тек 1 с нұсқасымен салыстырылды; сервиске бір vCPU бөлінді және JIT-компиляцияны ескеретін қыздыру қолданылды. Танымал кілттерде Redis SQL сұраныстарының 91 %-ын жойып, 1 с күту уақытында 99-перцентиль кідірісін 32–61 %-ға қысқартты, бірақ орташа өткізу қабілеті сенімді өзгерген жоқ (1000 пайдаланушыда қатынасы 0,99); біркелкі кілттерде өткізу қабілеті 22–34 %-ға төмендеді. CPU есебі мұны түсіндіреді: дерекқор сұраныстың CPU уақытының тек 32 %-ын тұтынды, ал кэш жолы JVM жұмысын көбейтті. Кідірісті 60 %-ға азайту және өткізу қабілетін үш есе арттыру туралы гипотезалар расталмады.

Түйінді сөздер: Spring Boot, Redis, cache-aside, REST API, жүктемелік тестілеу, кідіріс құйрығы, JIT-қыздыру.

ИССЛЕДОВАНИЕ ПРОИЗВОДИТЕЛЬНОСТИ SPRING BOOT REST API И ОЦЕНКА КЭШИРОВАНИЯ REDIS ПРИ ВЫСОКОЙ НАГРУЗКЕ

Serik Zholdygaliev*, Ravil Yermukhan, Kulush Askhat
Международный университет информационных технологий, Алматы, Казахстан
E-mail: sikozholdigaliev@gmail.com, r.yermukhan@gmail.com, finf0094@gmail.com
**Автор для корреспонденции*

Аннотация. Кэширование в Redis часто рекомендуется для REST API с преобладанием чтения, однако его эффект редко измеряется. Оценивается паттерн Redis cache-aside в API на Spring Boot 3.5 с PostgreSQL 16. В 75 рандомизированных парных прогонах вариант без кэша сравнивался с Redis при тайм-ауте 100 мс и 1 с при 100, 500 и 1000 виртуальных пользователях на «популярных» ключах, а на равномерных только с вариантом 1 с; сервису выделялся один vCPU, применялся JIT-прогрев. На популярных ключах Redis устранил 91 % SQL-запросов и при тайм-ауте 1 с сократил 99-й перцентиль задержки на 32–61 %, но средняя пропускная способность достоверно не изменилась (отношение 0,99 при 1000 пользователях); при

равномерной нагрузке она снизилась на 22–34 %. Учёт процессорного времени объясняет это: на базу данных приходилось лишь 32 % времени CPU на запрос, а путь через кэш добавлял работу JVM. Гипотезы о снижении задержки на 60 % и трёхкратном росте пропускной способности не подтвердились.

Ключевые слова: Spring Boot, Redis, cache-aside, REST API, нагрузочное тестирование, хвостовая задержка, прогрев JIT.

Introduction

Read-heavy REST APIs form the backbone of online catalogues, university portals and e-government services, where a single page view can trigger several lookups of products or records. As the number of concurrent users grows, the relational database behind such an API is usually the first shared resource to saturate, and response time increases accordingly. A widely recommended remedy is to place an in-memory key–value store in front of the database and to answer repeated reads from memory. Large deployments of this idea at Facebook (Nishtala et al., 2013; Berg et al., 2020) and Twitter (Yang et al., 2020) show that a cache is a standard architectural component rather than an optional optimisation.

In the Java ecosystem, Spring Boot is a common framework for building such APIs, and Redis is the cache most frequently placed next to it. Nevertheless, the expected benefit of adding Redis is often stated as a rule of thumb ("several times faster") instead of being measured for a concrete configuration. Therefore, before a team introduces an additional infrastructure component, it needs evidence about how large the gain is, under which load it appears and what the cache costs in resources and failure modes.

Several recent studies examine the performance of REST frameworks themselves. Godinho et al. (2024) compared equivalent .NET and Spring Boot APIs with a common test suite and showed that the outcome depends on the operation type and the load profile. Szewczyk and Skublewska-Paszkowska (2025) benchmarked five frameworks and reported that Spring Boot performed close to ASP.NET, whereas Serej et al. (2026) found that Ktor achieved lower response times and lower CPU usage than Spring Boot for the same PostgreSQL-backed reservation API. Similarly, Salunke and Ouda (2024) showed that the relative speed of PostgreSQL and MySQL depends on the data volume and the operation. These works measure framework or database overhead, but they do not isolate the contribution of a cache layer.

Studies that focus on caching, in turn, report favourable but context-dependent effects. Privalov and Stupina (2024) proposed Redis-based caching mechanisms to raise the efficiency of a Java web-oriented information system, and Kabakus and Kara (2017) compared in-memory databases and concluded that no single store is best for every operation. Production analyses explain why the benefit varies: the popularity of web requests follows Zipf-like distributions (Breslau et al., 1999), key–value workloads are strongly skewed (Atikoglu et al., 2012), and TTL settings and write intensity shape the working set of real cache clusters (Yang et al., 2020). Furthermore, the eviction policy influences both the miss ratio and the throughput of a cache (Zhang et al., 2024), and an application-level cache may return stale data unless consistency is managed explicitly (Ports et al., 2010).

Measuring such effects correctly is itself a methodological problem. Java performance varies between JVM invocations because of just-in-time compilation and garbage collection, so a single measurement can lead to a wrong conclusion (Georges et al., 2007); for this reason, Kalibera and Jones (2013) recommend repeated independent executions and effect sizes with confidence intervals. The load generator must also be chosen carefully: a closed generator, in which each virtual user waits for the previous response, keeps the number of requests in the system constant and can hide queueing that an open arrival process would reveal (Schroeder et al., 2006). Finally, mean values must be complemented by tail percentiles, because rare slow requests dominate the latency perceived by users at scale (Dean and Barroso, 2013).

Taken together, the reviewed literature confirms that caching can help, but it leaves three practical questions open for a typical Spring Boot service. First, how much does a Redis cache-aside layer change latency and throughput when the whole service runs on a small, CPU-constrained node? Second, how does the client-side Redis timeout interact with CPU saturation? Third, which share of the per-request cost can a cache actually remove? Within the reviewed sources, the measured effects are rarely decomposed into CPU time per component, so reported speed-ups are difficult to transfer to another system.

The aim of this study is to quantify the effect of Redis cache-aside on a Spring Boot REST API backed by PostgreSQL under high concurrency and to explain the observed effect with component-level measurements. The study answers three research questions:

– **RQ1** – How does Redis cache-aside change the mean and tail response time at 100, 500 and 1000 concurrent virtual users (VUs)?
– **RQ2** – How does it change successful throughput and the share of failed requests?
– **RQ3** – How do key locality, cache hit ratio, database calls and CPU time per request explain the differences?

Following the research plan prepared before the experiment, two quantitative hypotheses were tested for the primary condition (hot-key workload, 1000 VUs): **H1** – Redis reduces the mean response time by at least 60 %; **H2** – Redis increases successful throughput at least threefold. The novelty of the work lies in a controlled paired design with process-level CPU accounting, which links the measured effect to an Amdahl-type upper bound (Amdahl, 1967), in a JIT-aware warm-up protocol for a saturated single-core JVM, and in the explicit evaluation of the Redis client timeout as a failure mechanism under saturation.

Section summary. The introduction showed that Redis caching is widely used, yet its benefit depends on workload locality, configuration and the cost structure of a request. Consequently, the gain cannot simply be assumed and has to be measured under controlled conditions. The next section therefore explains how such a controlled experiment was designed and carried out.

Materials and methods

Research design

The study is a quantitative controlled experiment with randomised paired blocks. A block is one combination of workload, concurrency level and repetition; inside each block the same application was measured in every configuration in random order. As a result, slow drifts of the host, such as background activity of the virtual machine, cannot systematically favour one configuration. Three configurations formed the main independent variable:

– **B** (baseline) – every request reads PostgreSQL through JDBC;
– **R100** – Redis cache-aside with a 100 ms client command timeout, as fixed in the original research plan;
– **R1000** – the same cache with a 1000 ms timeout.

The R1000 configuration was added after a full-stack pilot and before the main data collection, because the pilot showed frequent Redis timeouts in R100 at high concurrency. Concurrency (100, 500 and 1000 VUs) and key locality (hot and uniform) were the other factors. Hypotheses H1 and H2 were evaluated for both cache configurations against B.

System under test

Figure 1a shows the testbed. The reference application was written for this study in Java 21 with Spring Boot 3.5.6 and exposes one endpoint, GET /api/products/{id}, which returns a product summary: name, category, price, version, a 256-character description, the number of reviews and the average rating. The summary is produced by one SQL statement that joins products with categories and aggregates the five reviews of the product through an index on reviews(product_id).

In configuration B the service calls the repository directly. In configurations R100 and R1000 it applies the cache-aside pattern shown in Figure 1b: Redis is queried first; on a hit the stored JSON is returned without SQL; on a miss the database result is stored with a time-to-live (TTL) of 300 s. If Redis does not answer within the timeout, the error is counted and the request falls back to PostgreSQL, so a cache fault never turns into an HTTP error. Redis was limited to 32 MiB with the allkeys-lru eviction policy (Redis Ltd., 2026), Tomcat used its default pool of 200 worker threads, HikariCP kept 20 database connections and the JVM heap was fixed at 512 MiB.

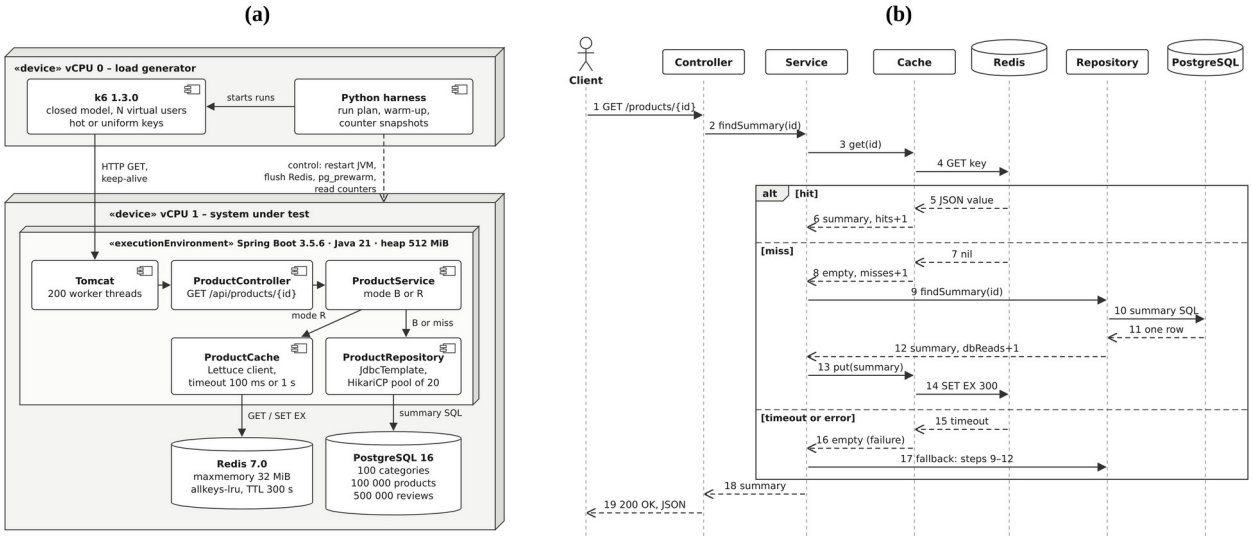


Figure 1 – UML diagrams of the system under test: (a) deployment diagram of the experimental testbed; (b) sequence diagram of the read path in the Redis configurations – hit, miss and timeout fallback (own elaboration; source files S1: diagrams/fig1_testbed_deployment.svg, diagrams/fig2_cache_aside_sequence.svg)

The implementation itself is compact (Figure 2). ProductService decides between the control path and the cache path, while ProductCache isolates all Redis calls and converts every exception into a counted cache failure. In addition, the load script chooses product identifiers with a deterministic pseudo-random generator seeded per VU, so each run can be replayed exactly.

(a) ProductService.java — cache-bench/src/main/java/kz/iitu/cachebench/service/ProductService.java

```
25    public Optional<ProductSummary> findSummary(long id) {
26        if (mode == CacheMode.BASELINE) {
27            return repository.findSummary(id);                // control condition
28        }
29        Optional<ProductSummary> cached = cache.get(id);      // 1. look in Redis
30        if (cached.isPresent()) {
31            return cached;                                    // 2a. hit: no SQL
32        }
33        Optional<ProductSummary> fromDb = repository.findSummary(id); // 2b. miss: PostgreSQL
34        fromDb.ifPresent(cache::put);                         // 3. populate with TTL
35        return fromDb;
36    }
```

(b) ProductCache.java — cache-bench/src/main/java/kz/iitu/cachebench/cache/ProductCache.java

```
31    public Optional<ProductSummary> get(long id) {
32        try {
33            String json = redis.opsForValue().get(KEY_PREFIX + id);
34            if (json == null) {
35                stats.miss();
36                return Optional.empty();
37            }
38            stats.hit();
39            return Optional.of(mapper.readValue(json, ProductSummary.class));
40        } catch (Exception e) {            // timeout, outage or unreadable value
41            stats.lookupFailure();
42            return Optional.empty();       // caller falls back to PostgreSQL
43        }
44    }
```

Figure 2 – Screenshots of the implementation in light mode: (a) cache-aside logic in ProductService; (b) fault-tolerant Redis lookup in ProductCache (own code; source file S2: cache-bench/src/main/java/kz/iitu/cachebench)

Dataset and workloads

The PostgreSQL database contains synthetic, deterministic data: 100 categories, 100 000 products and 500 000 reviews (99 MB). No personal data were used, so the study required no participant consent. Each response is about 0.39 kB of JSON. Following the YCSB practice of an explicit workload specification (Cooper et al., 2010), two key-selection workloads were defined. In the **hot** workload, 90 % of requests target the 1000 most popular products and 10 % target any of the 100 000 products, reflecting the skew reported for real key–value stores (Atikoglu et al., 2012). In the

uniform workload all products are equally likely; consequently, its potential working set (56 MiB of cached values) exceeds the 32 MiB Redis limit.

The load generator was Grafana k6 1.3.0 with a closed constant-VU executor and zero think time: each VU sends the next request as soon as the previous one completes (Grafana Labs, 2026). Every response was validated – HTTP 200, the requested identifier, the expected product name and five reviews – and anything else was counted as a failure. The client timeout was 5 s.

Testbed and procedure

Experiments ran on a cloud virtual machine with two vCPUs (Intel Xeon, 2.1 GHz), 7.8 GiB RAM, Ubuntu 24.04 and Linux kernel 6.18. To prevent the load generator from competing with the service, processes were pinned with taskset: k6 and the harness on vCPU 0, and Spring Boot, PostgreSQL 16.15 and Redis 7.0.15 on vCPU 1. The system under test therefore had a budget of exactly one vCPU, comparable to a small cloud instance.

Figure 3 summarises the procedure. Before every run the JVM was restarted, Redis was flushed, statistics were reset and the PostgreSQL tables and indexes were loaded into shared buffers with pg_prewarm, so that every configuration started from the same state. The warm-up deserves special attention. A first series of 51 runs used a continuous 20 s warm-up; however, diagnostics with jcmd showed that under continuous saturation of one vCPU about 600 methods were still waiting in the HotSpot JIT compilation queue, because the compiler threads received too little processor time, and the same JVM became about 2.5 times faster once the queue had drained. That series was therefore discarded (its data are kept in the supplementary materials), in line with the warning that JVM measurements depend on compilation state (Georges et al., 2007). In the final protocol, the warm-up consisted of 10 s load bursts separated by 3 s pauses and was repeated until the compilation queue contained at most ten methods (at most six bursts; in practice two to five were needed). A 30 s measurement window followed. The plan contained 15 blocks with three configurations for the hot workload and 15 blocks with two configurations for the uniform workload, that is, 75 runs and five paired repetitions per cell (Table 1).

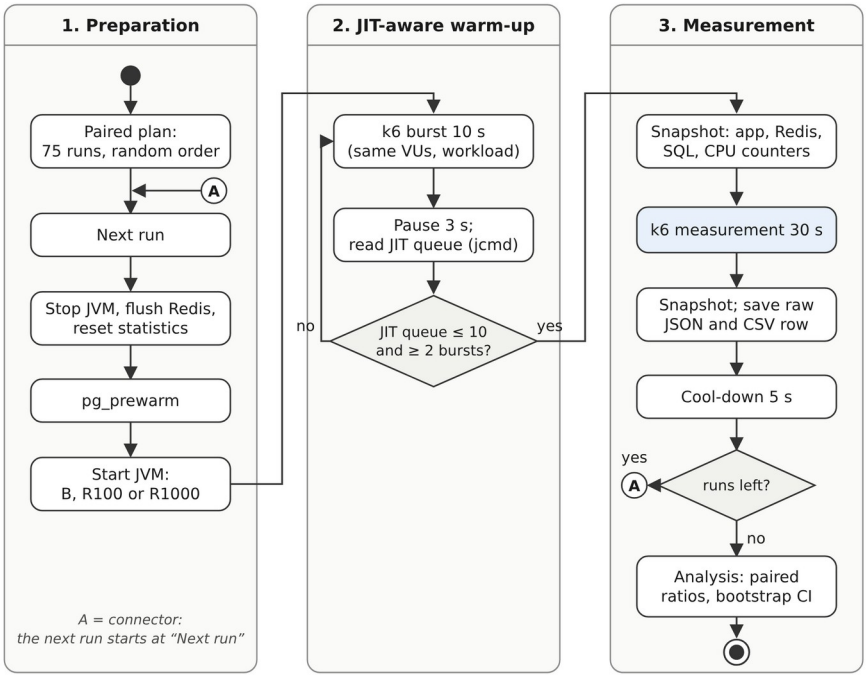


Figure 3 – Flowchart (UML activity diagram with swimlanes) of the measurement procedure with the JIT-aware warm-up; A is an on-page connector (own elaboration; source file S1: diagrams/fig4_procedure_flow.svg)

Table 1 – Factors, fixed settings and measured variables of the experiment

Element	Setting
Configuration (factor)	B – no cache; R100 – Redis cache-aside, 100 ms command timeout; R1000 – Redis cache-aside, 1000 ms timeout
Concurrency (factor)	100, 500 and 1000 virtual users, closed model, zero think time

Element	Setting
Key locality (factor)	hot – 90 % of requests to products 1…1000, 10 % to any of 100 000; uniform – all products equally likely
Replication	5 randomised paired blocks per cell; 75 runs (45 hot, 30 uniform)
One run	JVM restart, Redis flush, pg_prewarm, warm-up of 2–6 bursts of 10 s until the JIT queue is empty, 30 s measurement, 5 s cool-down
Application	Spring Boot 3.5.6, Java 21, heap 512 MiB, Tomcat 200 threads, HikariCP 20 connections
Cache	Redis 7.0.15, maxmemory 32 MiB, allkeys-lru, TTL 300 s, Lettuce client (one shared connection)
Database	PostgreSQL 16.15, shared_buffers 256 MB; 100 categories, 100 000 products, 500 000 reviews
Hardware	2 vCPU Intel Xeon 2.1 GHz, 7.8 GiB RAM; vCPU 0 – k6, vCPU 1 – system under test
Measured variables	mean, p95, p99 latency; successful throughput; failure share; hit ratio; SQL calls and CPU ms per request

Source: own elaboration; settings are taken from the configuration files in supplementary files S2–S4.

Metrics and statistical analysis

For each run the following metrics were computed: the mean, median, 95th and 99th percentiles of response time over all completed requests; successful throughput, defined as validated responses completed inside the 30 s window divided by 30 s; the failure share; the cache hit ratio (hits divided by hits, misses and lookup failures); SQL statements per request from pg_stat_statements (PostgreSQL Global Development Group, 2026); and CPU milliseconds per request for the JVM, PostgreSQL and Redis processes, obtained from Linux /proc counters. A run was the statistical unit, and individual requests were not treated as independent observations (Kalibera and Jones, 2013).

For each block the ratio of the cache configuration to B was calculated for mean latency and for throughput, and 10 000 bootstrap resamples of the five blocks (Efron, 1979) gave percentile confidence intervals. For the primary condition two endpoints were tested, so Bonferroni-adjusted 97.5 % intervals were used: H1 is supported only if the upper bound of the latency ratio is at most 0.40, and H2 only if the lower bound of the throughput ratio is at least 3.0. As an internal consistency check, Little's law $N = X \cdot R$ (Little, 1961) was used to compare the number of VUs N with the product of measured throughput X and mean response time R.

Section summary. The method combines a controlled paired experiment with resource accounting: the configurations differ only in the cache path and the timeout, while the data, hardware, warm-up and request sequences are identical. Because every run also records hits, SQL calls and CPU time, the results can be explained and not only reported. The next section presents these measurements.

Results

Correctness and stability of the measurements

All 75 planned runs were completed, and 7.5 million responses were validated. Only 62 requests failed, all of them because the 5 s client timeout expired; no response contained a wrong body, and the failure share never exceeded 0.012 % in any run. During the measurement windows vCPU 1 was 87.2–100 % busy, so the service was CPU-bound at every concurrency level. The load generator usually had spare capacity; however, in 5 exceptionally fast runs it used more than 90 % of vCPU 0, so their throughput is a lower bound. Little's law held closely: in every individual run, the product of throughput and mean response time equalled 97–100 % of the configured number of VUs (products of five-run averages, as in Table 2, can be slightly higher because throughput varied between runs). At the same time, repeated runs of the same configuration differed considerably: the coefficient of variation of throughput within a cell ranged from 13 % to 46 %, even though every run used a fresh JVM, identical data and the same warm-up rule.

Latency and throughput across concurrency

Figure 4 shows mean response time and throughput for all cells. On the hot workload the three configurations overlapped at every concurrency level: the throughput ratio of a cache configuration to B ranged from 0.82 to 1.42, and every 95 % confidence interval included 1. For example, at 500 VUs R1000 reached 3 840 requests per second against 3 018 for B (ratio 1.27, CI 0.92–1.73). Because the

node was saturated, throughput hardly changed with the number of VUs, while mean latency grew roughly in proportion to it, as expected for a closed system.

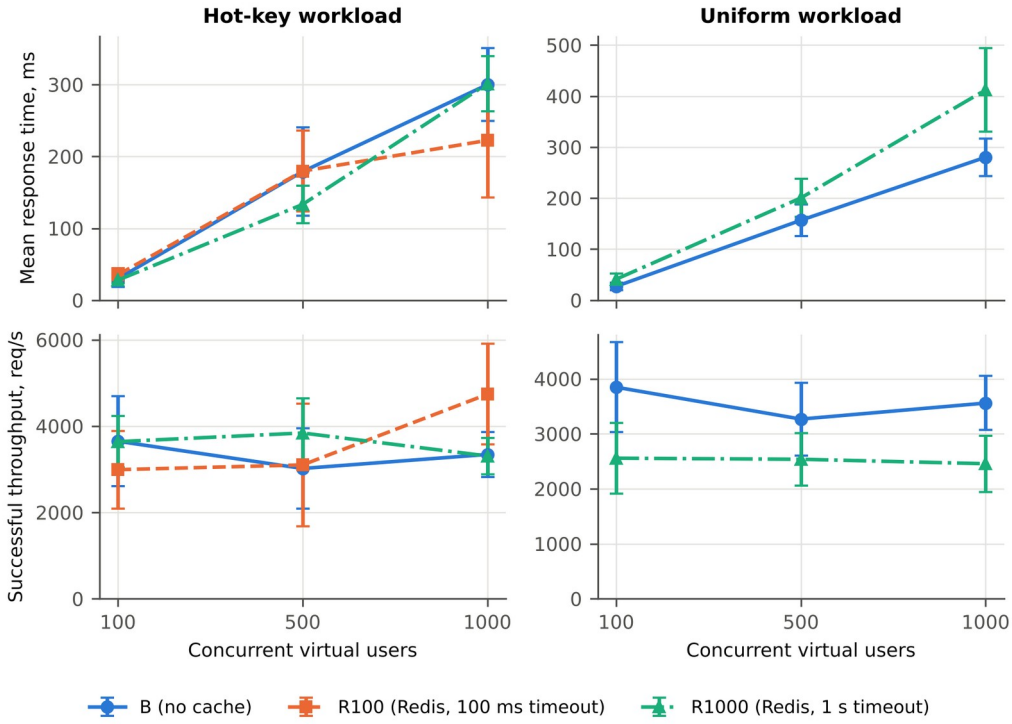


Figure 4 – Mean response time and successful throughput versus concurrency; points are means of five runs, bars show ± SD (own measurements; source files S5: results/derived_runs.csv and S6: analysis/make_charts.py)

On the uniform workload, by contrast, the cache consistently slowed the service down. R1000 delivered 0.66, 0.78 and 0.69 of the baseline throughput at 100, 500 and 1000 VUs, and all three confidence intervals lay below 1; the mean response time rose accordingly, for instance from 280.2 ms to 412.2 ms at 1000 VUs. The tail of the distribution behaved differently from the mean. With R1000 the 99th percentile on the hot workload fell from 733 ms to 283 ms at 500 VUs and from 859 ms to 534 ms at 1000 VUs; the corresponding 95th-percentile ratios were 0.52 and 0.78, with confidence intervals entirely below 1.

Primary condition and test of the hypotheses

Table 2 summarises the pre-specified primary condition. Compared with B, R1000 changed neither the mean response time (ratio 1.00, 97.5 % CI 0.89–1.13) nor successful throughput (ratio 0.99, CI 0.89–1.11), although it removed 91 % of SQL statements and shortened the 99th percentile by 38 %. R100 showed a higher point estimate of throughput (1.42, CI 0.99–1.85), but this mean was driven by three exceptionally fast runs in which the generator itself approached saturation, and its standard deviation was the largest in the study. Because the upper bound of every latency ratio is well above 0.40 and the lower bound of every throughput ratio is well below 3.0, neither H1 nor H2 is supported for either cache configuration.

Table 2 – Primary condition (hot-key workload, 1000 VUs): mean ± SD over five runs and paired effects

Metric	B	R100	R1000
Mean response time, ms	299.9 ± 50.8	222.3 ± 79.5	301.1 ± 38.1
p95 / p99 response time, ms	579 / 859	416 / 617	451 / 534
Successful throughput, req/s	3 343 ± 518	4 737 ± 1 167	3 308 ± 422
Failed requests, %	0.002	0.000	0.000
Cache hit ratio, %	–	91	91
Redis lookups failed (timeout), %	–	0.8	0.0
SQL statements per request	1.00	0.09	0.09
CPU time per request, ms	0.30	0.21	0.30

Metric	B	R100	R1000
Latency ratio to B [97.5 % CI]	1	0.74 [0.53; 1.11]	1.00 [0.89; 1.13]
Throughput ratio to B [97.5 % CI]	1	1.42 [0.99; 1.85]	0.99 [0.89; 1.11]
H1: latency ratio ≤ 0.40	–	not supported (upper bound 1.11 vs ≤ 0.40)	not supported (upper bound 1.13 vs ≤ 0.40)
H2: throughput ratio ≥ 3.0	–	not supported (lower bound 0.99 vs ≥ 3.0)	not supported (lower bound 0.89 vs ≥ 3.0)

Source: own measurements (supplementary file S5: results/runs.csv, results/paired_effects.csv). CI – bootstrap percentile interval over five paired blocks, 10 000 resamples.

Mechanism: hit ratio, database calls and CPU cost

The cache worked as intended at the data level: the hit ratio was 91 % on the hot workload and 50 % on the uniform one, where Redis reached its 32 MiB limit and evicted 241 770 keys, and SQL statements per request fell from 1.00 to 0.09 and 0.50, respectively. Figure 5a shows where the processor time went. In B, a request cost on average 0.32 ms of CPU time over all 15 hot-workload runs (0.30 ms in the primary condition, Table 2), of which PostgreSQL accounted for 0.100 ms (32 %) and the JVM for 0.21 ms. With R1000, PostgreSQL time dropped to 0.018 ms, but the JVM time rose to 0.26 ms, so the total fell only to 0.28 ms. On the uniform workload, misses made the cache path clearly more expensive (0.41 ms versus 0.28 ms). A supplementary diagnostic run showed that the JVM performed 4.0–4.1 context switches per request in R1000 versus 2.4–2.5 in B, because each Redis call is handed over to the client's network thread and back. Figure 5b confirms that throughput is governed by this CPU cost: the value predicted as available CPU time divided by CPU time per request matches the measured throughput within 2 % for every run.

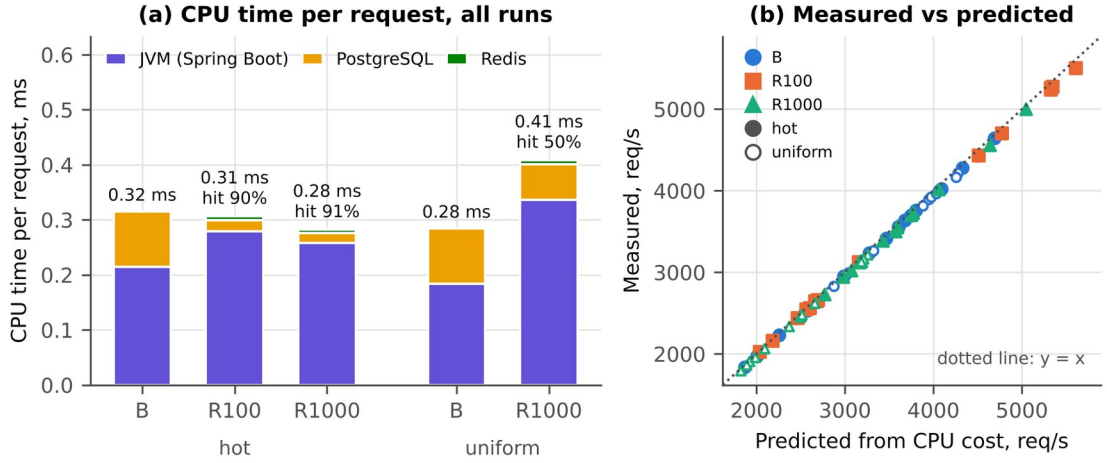


Figure 5 – Mechanism of the effect: (a) CPU time per request by process, mean of all 15 runs of each configuration and workload, with mean hit ratio; (b) measured throughput versus the value predicted from CPU cost, one point per run (own measurements; source files S5 and S6)

Section summary. The measurements are valid and internally consistent: the service was CPU-bound, failures were negligible and Little's law held. Redis removed most database calls and shortened the latency tail, yet on a skewed workload it did not change mean latency or throughput reliably, and on a uniform workload it reduced throughput by roughly a quarter to a third. The CPU accounting already points to the reason, which the next section discusses together with earlier studies.

Discussion

The central finding is that, on a single CPU-bound node, Redis cache-aside moved work from PostgreSQL to the JVM instead of removing it. In the baseline, the database accounted for only about 32 % of CPU time per request, whereas HTTP parsing in Tomcat, object mapping and JSON serialisation had to be performed regardless of where the data came from. According to Amdahl's law (Amdahl, 1967), eliminating a fraction f of the work yields a speed-up of at most 1/(1 – f); with f ≈ 0.32 the bound is about 1.47×, and it would be reached only if a cache hit cost nothing. In reality a hit is not free: the Lettuce client hands each command to its network thread and back, which increased the context switches of the JVM by about two thirds, and the cached JSON has to be deserialised before

the response is serialised again. These costs consumed most of the saving on the hot workload and exceeded it on the uniform one. Therefore, a threefold gain (H2) would require a database that dominates the request cost, such as the remote, heavily loaded back ends described by Nishtala et al. (2013) and Berg et al. (2020), rather than an indexed primary-key lookup on a co-located PostgreSQL instance.

Moreover, in a closed system latency and throughput are linked. With the number of VUs N fixed, Little's law gives $R = N / X$, so a 60 % reduction of mean latency (H1) at fixed concurrency is equivalent to a 2.5-fold increase of throughput, and both hypotheses fail for the same reason. This also means that the latency values at 500 and 1000 VUs mainly reflect queueing in front of a saturated processor rather than the speed of an individual database or cache access.

The second finding concerns the timeout. With 200 Tomcat threads competing for one vCPU, replies from Redis were sometimes delivered later than 100 ms although Redis itself used little CPU; in R100 up to 7.1 % of lookups in a run timed out. Every timeout raised an exception, triggered a fallback query and often a failed write. In addition, the JIT compilation queue after the measurement window averaged 398 methods for R100 at 500 VUs (at most 554), compared with a mean of at most 65 for any other configuration and load, which indicates that the rare exception paths forced recompilation in the middle of the measurement. This mechanism is consistent with the counters but was not traced at thread level. Consequently, R100 was the least predictable configuration; a timeout that is safe on an idle machine becomes a source of extra work under saturation and should be derived from the tail latency measured at peak load.

The third finding is the role of locality. The hot workload reused 1000 keys, so the hit ratio reached about 91 %. Under the uniform workload the potential working set (56 MiB) exceeded the Redis memory limit, the hit ratio stayed near 50 %, and every miss paid for a Redis lookup, the SQL query and a Redis write. This agrees with production evidence that the benefit of caching depends on request skew and working-set size (Breslau et al., 1999; Atikoglu et al., 2012; Yang et al., 2020).

Nevertheless, the cache had two practical benefits. First, it shortened the tail: in B, 200 worker threads competed for 20 JDBC connections, so some requests waited in the pool queue for a long time, whereas cache hits bypassed the pool. This supports the recommendation of Dean and Barroso (2013) to evaluate tail latency separately from the mean. Second, it removed about nine of every ten SQL statements, which matters when one database is shared by many application instances. Table 3 places these results in the context of earlier work.

Table 3 – Comparison of the present results with related studies

Study	Setting and finding	Relation to the present study
Privalov and Stupina (2024)	Java/Spring Boot web system with MySQL and Redis; tests with and without caching showed that well-chosen caching strategies improve performance.	Different outcome on a one-vCPU node: no reliable mean gain on hot keys (throughput ratio 0.99), a loss on uniform keys; the benefit is limited to database offload and tail latency.
Godinho et al. (2024); Szewczyk and Skublewska-Paszkowska (2025)	Framework comparisons of REST APIs: results depend on operation and load; Spring Boot is close to ASP.NET.	Consistent: framework work in the JVM (≈ 68 % of CPU per request in B) dominates the cost of a simple read.
Serej et al. (2026)	Ktor showed lower response time and CPU usage than Spring Boot for the same PostgreSQL-backed API.	Supports the finding that per-request JVM overhead, not the database, limits throughput on a small node.
Nishtala et al. (2013); Berg et al. (2020)	Large production caches at Facebook remove expensive remote database reads and enable very high request rates.	Large gains require an expensive back end; with a cheap co-located query the Amdahl bound is only ≈ 1.47×.
Breslau et al. (1999); Atikoglu et al. (2012); Yang et al. (2020)	Real request popularity is skewed; TTL and working-set size shape cache behaviour.	Confirmed: hit ratio 91 % for hot keys versus 50 % for uniform keys decides whether the cache helps or harms.
Dean and Barroso (2013); Georges et al. (2007)	Tail latency must be managed separately from the mean; Java measurements vary between JVM invocations.	Observed: p99 reduced by up to 61 % while the mean did not change; throughput of identical runs varied by up to 46 %.

Source: own elaboration based on the cited publications.

Finally, the experiment yielded a methodological lesson. Under continuous saturation of one vCPU, the HotSpot compiler threads were starved, and a first series of runs measured an incompletely compiled JVM. Even after the JIT-aware warm-up, throughput of identical configurations varied by up to 46 % between JVM invocations, which confirms the warning of Georges et al. (2007) and

explains why single measurements of Java services can be misleading. Several limitations follow. All server components shared one vCPU; with more cores, or with PostgreSQL on a separate host, the database share of the request cost, and hence the Amdahl bound, would change. The closed zero-think-time model keeps the number of outstanding requests fixed, whereas an open arrival process can create longer queues (Schroeder et al., 2006). Five 30 s repetitions per cell give wide confidence intervals, and in the fastest runs the generator approached its own limit. Finally, the workload was read-only and synthetic, so invalidation and consistency costs (Ports et al., 2010) were not measured. Two hypotheses for further work follow: the benefit of the cache grows with the database share of the request cost, and caching the serialised response or using fewer worker threads reduces the JVM overhead of a hit.

Section summary. The discussion showed that the absence of a mean speed-up is not an anomaly but a consequence of the request cost structure (Amdahl's law), of the closed load model (Little's law) and of the extra JVM work on the cache path, while the timeout and key locality decide whether the cache helps or harms. The final section answers the research questions and formulates practical recommendations.

Conclusion

This study measured the effect of Redis cache-aside on a Spring Boot REST API backed by PostgreSQL in 75 randomised paired runs, with the system under test limited to one vCPU and a JIT-aware warm-up. The answers to the research questions are as follows.

- **RQ1.** On the hot-key workload, the mean response time did not change reliably (all confidence intervals included 1), but with a 1 s timeout the 99th percentile decreased by up to 61 %; on the uniform workload the mean response time increased by 28–52 %.
- **RQ2.** Successful throughput did not change reliably on the hot workload and decreased by 22–34 % on the uniform workload; failed requests did not exceed 0.012 % in any run, whereas with a 100 ms timeout up to 7.1 % of Redis lookups failed under saturation and made performance unstable.
- **RQ3.** The cache removed 91 % of SQL statements on hot keys, but PostgreSQL accounted for only about 32 % of the request cost (Amdahl bound ≈ 1.47×), and the cache path added JVM work, so the net CPU saving was small on hot keys and negative on uniform keys.

Consequently, hypotheses H1 (at least 60 % latency reduction) and H2 (at least threefold throughput) are not supported. This negative result is useful in practice: before introducing Redis, developers should measure the database share of the request cost and the locality of keys, set the client timeout from the tail latency measured under peak load, and warm up and repeat JVM measurements carefully. Future work will repeat the experiment with the database on a separate host, with more cores, heavier queries, cached serialised responses and an open arrival-rate workload, and will add write traffic to study invalidation and consistency.

References

Amdahl, G.M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the AFIPS Spring Joint Computer Conference*, 483–485. DOI: https://doi.org/10.1145/1465482.1465560 [In Eng.]

Atikoglu, B., Xu, Y., Frachtenberg, E., Jiang, S., Paleczny, M. (2012). Workload analysis of a large-scale key-value store. *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems*, 53–64. DOI: https://doi.org/10.1145/2254756.2254766 [In Eng.]

Berg, B., Berger, D.S., McAllister, S., Grosof, I., Gunasekar, S., Lu, J., Uhlar, M., Carrig, J., Beckmann, N., Harchol-Balter, M., Ganger, G.R. (2020). The CacheLib caching engine: Design and experiences at scale. *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 753–768. URL: https://www.usenix.org/conference/osdi20/presentation/berg [In Eng.]

Breslau, L., Cao, P., Fan, L., Phillips, G., Shenker, S. (1999). Web caching and Zipf-like distributions: Evidence and implications. *Proceedings of IEEE INFOCOM '99*, 1, 126–134. DOI: https://doi.org/10.1109/INFCOM.1999.749260 [In Eng.]

Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R. (2010). Benchmarking cloud serving systems with YCSB. *Proceedings of the 1st ACM Symposium on Cloud Computing*, 143–154. DOI: https://doi.org/10.1145/1807128.1807152 [In Eng.]

Dean, J., Barroso, L.A. (2013). The tail at scale. *Communications of the ACM*, 56(2), 74–80. DOI: https://doi.org/10.1145/2408776.2408794 [In Eng.]

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1–26. DOI: https://doi.org/10.1214/aos/1176344552 [In Eng.]

Georges, A., Buytaert, D., Eeckhout, L. (2007). Statistically rigorous Java performance evaluation. *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, 57–76. DOI: https://doi.org/10.1145/1297027.1297033 [In Eng.]

Godinho, A., Rosado, J., Sá, F., Cardoso, F. (2024). Performance comparison of RESTful web APIs using a test suite: .NET vs. Java Spring Boot. *Journal of Software and Systems Development*, 2024, Article 478010. DOI: https://doi.org/10.5171/2024.478010 [In Eng.]

Grafana Labs. (2026). Open and closed models. *Grafana k6 documentation*. URL: https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/ (accessed: 18.10.2026) [In Eng.]

Kabakus, A.T., Kara, R. (2017). A performance evaluation of in-memory databases. *Journal of King Saud University – Computer and Information Sciences*, 29(4), 520–525. DOI: https://doi.org/10.1016/j.jksuci.2016.06.007 [In Eng.]

Kalibera, T., Jones, R. (2013). Rigorous benchmarking in reasonable time. *Proceedings of the 2013 International Symposium on Memory Management (ISMM '13)*, 63–74. DOI: https://doi.org/10.1145/2491894.2464160 [In Eng.]

Little, J.D.C. (1961). A proof for the queuing formula: L = λW. *Operations Research*, 9(3), 383–387. DOI: https://doi.org/10.1287/opre.9.3.383 [In Eng.]

Nishtala, R., Fugal, H., Grimm, S., Kwiatkowski, M., Lee, H., Li, H.C., McElroy, R., Paleczny, M., Peek, D., Saab, P., Stafford, D., Tung, T., Venkataramani, V. (2013). Scaling Memcache at Facebook. *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, 385–398. URL: https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala [In Eng.]

Ports, D.R.K., Zhang, I., Madden, S., Liskov, B. (2010). Transactional consistency and automatic management in an application data cache. *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. URL: https://www.usenix.org/conference/osdi10/transactional-consistency-and-automatic-management-application-data-cache [In Eng.]

PostgreSQL Global Development Group. (2026). pg_stat_statements – track statistics of SQL planning and execution. *PostgreSQL 16 documentation*. URL: https://www.postgresql.org/docs/16/pgstatstatements.html (accessed: 18.10.2026) [In Eng.]

Privalov, M.V., Stupina, M.V. (2024). Improving web-oriented information systems efficiency using Redis caching mechanisms. *Indonesian Journal of Electrical Engineering and Computer Science*, 33(3), 1667–1675. DOI: https://doi.org/10.11591/ijeecs.v33.i3.pp1667-1675 [In Eng.]

Redis Ltd. (2026). Key eviction. *Redis documentation*. URL: https://redis.io/docs/latest/develop/reference/eviction/ (accessed: 18.10.2026) [In Eng.]

Salunke, S.V., Ouda, A. (2024). A performance benchmark for the PostgreSQL and MySQL databases. *Future Internet*, 16(10), 382. DOI: https://doi.org/10.3390/fi16100382 [In Eng.]

Schroeder, B., Wierman, A., Harchol-Balter, M. (2006). Open versus closed: A cautionary tale. *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 06)*. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale [In Eng.]

Serej, M., Kopciński, K., Smołka, J. (2026). Comparative performance analysis of Spring Boot and Ktor for a ticket reservation REST API on the JVM. *Journal of Computer Sciences Institute*, 39, 183–187. DOI: https://doi.org/10.35784/jcsi.9586 [In Eng.]

Szewczyk, M., Skublewska-Paszkowska, M. (2025). Performance comparison of development frameworks in selected environments in REST API architecture. *Journal of Computer Sciences Institute*, 35, 121–128. DOI: https://doi.org/10.35784/jcsi.7041 [In Eng.]

Yang, J., Yue, Y., Rashmi, K.V. (2020). A large scale analysis of hundreds of in-memory cache clusters at Twitter. *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 191–208. URL: https://www.usenix.org/conference/osdi20/presentation/yang [In Eng.]

Zhang, Y., Yang, J., Yue, Y., Vigfusson, Y., Rashmi, K.V. (2024). SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches. *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 1229–1246. URL: https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo [In Eng.]

Supplementary materials

All visual elements and numbers of the article were produced by the authors from the files of the attached archive MT1_sources.zip: **S1** – diagrams/ (SVG sources of Figures 1a, 1b and 3, editable in draw.io or Inkscape); **S2** – cache-bench/ (Spring Boot source code, Figure 2); **S3** – bench/ (k6 script,

benchmark harness, Redis configuration, Table 1); **S4** – db/ (PostgreSQL schema and data generator); **S5** – results/ (run plan, raw data of all 75 runs, k6 summaries, derived tables; Tables 2–3, Figures 4–5), together with the discarded first series and the JIT and context-switch diagnostics; **S6** – analysis/ (statistics, chart and screenshot scripts); **S7** – figures/ (rendered figures and an extra screenshot of the k6 key-selection algorithm).

Information about the authors

Serik Zholdygaliev – student of the educational programme "Information Systems", International Information Technology University, Almaty, Kazakhstan; e-mail: sikozholdigaliev@gmail.com.

Ravil Yermukhan – student of the educational programme "Information Systems", International Information Technology University, Almaty, Kazakhstan; e-mail: r.yermukhan@gmail.com.

Kulush Askhat – student of the educational programme "Information Systems", International Information Technology University, Almaty, Kazakhstan; e-mail: finf0094@gmail.com.